

RetryableCallback使用手册

由 朱明创建, 最后修改于八月 30, 2023 49 views since 29 Aug 2023

-
- [一、FlinkKafkaProducer的构造函数使用](#)
- [二、实际发生函数invoke的差异](#)
 - [1.其中第9行到47行是keyedSchema，该方法不建议使用，但第31行对于使用该方式并且有flinkKafkaPartitioner的方式也进行了坏partition处理。](#)
 - [2.第44行，如果没有partitioner则会转到86行再处理。](#)
 - [3.第50行到67行为SerializationSchema方式，同样方式屏蔽坏partition。](#)
 - [4.如果是KafkaSerializationSchema，那么也是进入86行再处理](#)
- [三、有关kafka参数和RetryableProducerCallback参数规范](#)
 - [flush注意事项](#)

一、FlinkKafkaProducer的构造函数使用

目前Flink **1.13.6** 以及最新的flink-connector-kafka **V3.0** 主要构造函数为下面两大类。

第一类为**SerializationSchema**构造函数，第二类为**KafkaSerializationSchema**方式，**KeyedSerializationSchema**已经**Deprecated**。其中目前所有项目使用的是第二种。

```

1
2 // 第一类SerializationSchema 构造函数使用
3 public FlinkKafkaProducer(String topicId, SerializationSchema<IN> serializationSchema, Properties producerConfig) {
4     this(
5         topicId,
6         serializationSchema,
7         producerConfig,
8         Optional.of(new FlinkFixedPartitioner<>()));
9     }
10
11 public FlinkKafkaProducer(String topicId, SerializationSchema<IN> serializationSchema, Properties producerConfig,
12     Optional<FlinkKafkaPartitioner<IN>> customPartitioner) {
13     this(
14         topicId,
15         serializationSchema,
16         producerConfig,
17         customPartitioner.orElse(null),
18         Semantic.AT_LEAST_ONCE,
19         DEFAULT_KAFKA_PRODUCERS_POOL_SIZE);
20     }
21
22 public FlinkKafkaProducer(String topicId, SerializationSchema<IN> serializationSchema, Properties producerConfig,
23     @Nullable FlinkKafkaPartitioner<IN> customPartitioner, FlinkKafkaProducer.Semantic semantic, int kafkaProducersPo
24     this(
25         topicId,
26         null,
27         null,
28         new KafkaSerializationSchemaWrapper<>(topicId, customPartitioner, false, serializationSchema),
29         producerConfig,
30         semantic,
31         kafkaProducersPoolSize);
32     }
33
34 // 第二类KafkaSerializationSchema 构造函数使用, 目前主要使用方式
35 public FlinkKafkaProducer(String defaultTopic, KafkaSerializationSchema<IN> serializationSchema, Properties producerConf
36     this(
37         defaultTopic,
38         serializationSchema,
39         producerConfig,
40         semantic,
41         DEFAULT_KAFKA_PRODUCERS_POOL_SIZE);
42     }
43
44 public FlinkKafkaProducer(String defaultTopic, KafkaSerializationSchema<IN> serializationSchema, Properties producer
45     ,int kafkaProducersPoolSize) {
46     this(
47         defaultTopic,
48         null,
49         null, /* keyed schema and FlinkKafkaPartitioner */
50         serializationSchema,
51         producerConfig,
52         semantic,
53         kafkaProducersPoolSize);
54     }

```

最终指向该私有构造函数，其中主要差异为kafkaSchema参数的类型。

```

private FlinkKafkaProducer(
    String defaultTopic,
    KeyedSerializationSchema<IN> keyedSchema,
    FlinkKafkaPartitioner<IN> customPartitioner,
    KafkaSerializationSchema<IN> kafkaSchema,
    Properties producerConfig,
    FlinkKafkaProducer.Semantic semantic,
    int kafkaProducersPoolSize)

```

具体反映到使用层面，可以使用下面两种方式，作为任务的KafkaProducer的创建函数。

```

1
2 // 目前项目主要使用的模式，精简了一些参数的设置，数据对象为KafkaSerializationSchema
3 public static FlinkKafkaProducer<byte[]> createKafkaProducer(Properties kafkaProps, String outputTopic, FlinkKafkaProduc
4     return new FlinkKafkaProducer<byte[]> sink = new FlinkKafkaProducer<>(outputTopic,
5         (KafkaSerializationSchema<byte[]>) (element, timestamp) -> new ProducerRecord<>(outputTopic, null /* robin p

```

```

6         kafkaProps,
7         semantic);
8     }
9     // 该构造函数会将schema包装为KafkaSerializationSchemaWrapper
10    public static FlinkKafkaProducer<byte[]> createKafkaProducer(Properties kafkaProps, String outputTopic, FlinkKafkaProduc
11        return new FlinkKafkaProducer<>(outputTopic,
12            (SerializationSchema<byte[]>) (element) -> element,
13            kafkaProps,
14            partitioner,
15            semantic,
16            5
17    );

```

二、实际发生函数invoke的差异

实际写入kafka流程如下。主要有下面4个执行路径。

- 1.其中第9行到47行是keyedSchema，该方法不建议使用，但第31行对于使用该方式并且有flinkKafkaPartitioner的方式也进行了坏partition处理。
- 2.第44行，如果没有partitioner则会转到86行再处理。
- 3.第50行到67行为SerializationSchema方式，同样方式屏蔽坏partition。
- 4.如果是KafkaSerializationSchema，那么也是进入86行再处理

```

1    @Override
2
3    public void invoke(
4        FlinkKafkaProducer.KafkaTransactionState transaction, IN next, Context context)
5        throws FlinkKafkaException {
6        checkErroneous();
7
8        ProducerRecord<byte[], byte[]> record;
9        // 不建议使用keyedSchema
10       if (keyedSchema != null) {
11           byte[] serializedKey = keyedSchema.serializeKey(next);
12           byte[] serializedValue = keyedSchema.serializeValue(next);
13           String targetTopic = keyedSchema.getTargetTopic(next);
14           if (targetTopic == null) {
15               targetTopic = defaultTopicId;
16           }
17
18           Long timestamp = null;
19           if (this.writeTimestampToKafka) {
20               timestamp = context.timestamp();
21           }
22
23           int[] partitions = topicPartitions.getAvailablePartitions(targetTopic);
24           if (null == partitions) {
25               partitions = getPartitionsByTopic(targetTopic, transaction.producer);
26               topicPartitions.addPartitionsForTopic(targetTopic, partitions);
27           } else if (partitions.length == 0) {
28               throw new RuntimeException("We have no healthy partitions for topic=" + targetTopic + "!");
29           }
30
31           if (flinkKafkaPartitioner != null) {
32               record =
33                   new ProducerRecord<>(
34                       targetTopic,
35                       flinkKafkaPartitioner.partition(
36                           next,
37                           serializedKey,
38                           serializedValue,
39                           targetTopic,
40                           partitions),
41                           timestamp,
42                           serializedKey,
43                           serializedValue);
44           } else {
45               record =
46                   new ProducerRecord<>(
47                       targetTopic, null, timestamp, serializedKey, serializedValue);
48           }
49       } else if (kafkaSchema != null) {
50           // 上述SerializationSchema构造方式

```

```

51     if (kafkaSchema instanceof KafkaContextAware) {
52         @SuppressWarnings("unchecked")
53         KafkaContextAware<IN> contextAwareSchema = (KafkaContextAware<IN>) kafkaSchema;
54
55         String targetTopic = contextAwareSchema.getTargetTopic(next);
56         if (targetTopic == null) {
57             targetTopic = defaultTopicId;
58         }
59         int[] partitions = topicPartitions.getAvailablePartitions(targetTopic);
60
61         if (null == partitions) {
62             partitions = getPartitionsByTopic(targetTopic, transaction.producer);
63             topicPartitions.addPartitionsForTopic(targetTopic, partitions);
64         } else if (partitions.length == 0) {
65             throw new RuntimeException("We have no healthy partitions for topic=" + targetTopic + "!");
66         }
67
68         contextAwareSchema.setPartitions(partitions);
69     }
70     // 上述KafkaSerializationSchema直接流程
71     record = kafkaSchema.serialize(next, context.timestamp());
72
73     String targetTopic = record.topic();
74     int[] partitions = topicPartitions.getAvailablePartitions(targetTopic);
75     if (null == partitions) {
76         partitions = getPartitionsByTopic(targetTopic, transaction.producer);
77         topicPartitions.addPartitionsForTopic(targetTopic, partitions);
78     } else if (partitions.length == 0) {
79         throw new RuntimeException("We have no healthy partitions for topic=" + targetTopic + "!");
80     }
81 } else {
82     throw new RuntimeException(
83         "We have neither KafkaSerializationSchema nor KeyedSerializationSchema, this"
84         + "is a bug.");
85 }
86
87 if (record.partition() == null && topicPartitions.getBadPartitionInfo() != null) {
88     if (defaultFlinkKafkaPartitioner == null) {
89         defaultFlinkKafkaPartitioner = new FlinkFixedPartitioner<>();
90         defaultFlinkKafkaPartitioner.open(getRuntimeContext().getIndexOfThisSubtask(), getRuntimeContext().getNu
91     }
92     int[] partitions = topicPartitions.getAvailablePartitions(record.topic());
93     int partition = defaultFlinkKafkaPartitioner.partition(
94         next,
95         record.key(),
96         record.value(),
97         record.topic(),
98         partitions
99     );
100     record = new ProducerRecord<>(
101         record.topic(),
102         partition,
103         record.timestamp(),
104         record.key(),
105         record.value(),
106         record.headers()
107     );
108 }
109
110
111 pendingRecords.incrementAndGet();
112 FlinkKafkaInternalProducer<byte[], byte[]> producer = transaction.getProducer();
113 switch (semantic) {
114     // EXACTLY_ONCE cannot retry send failed message by using current producer instance, Once failed this produc
115     case EXACTLY_ONCE:
116         producer.send(record, callback);
117         break;
118     case AT_LEAST_ONCE:
119     case NONE:
120         RetryableProducerCallback<IN> producerCallback = new RetryableProducerCallback<>(
121             next,
122             producer,
123             actualCallbackRetryTimes,
124             flinkKafkaPartitioner,
125             record,
126             getRuntimeContext(),
127             retryMeter,
128             topicPartitions,

```

```

129         callback);
130     producer.send(record, producerCallback);
131     break;
132 }

```

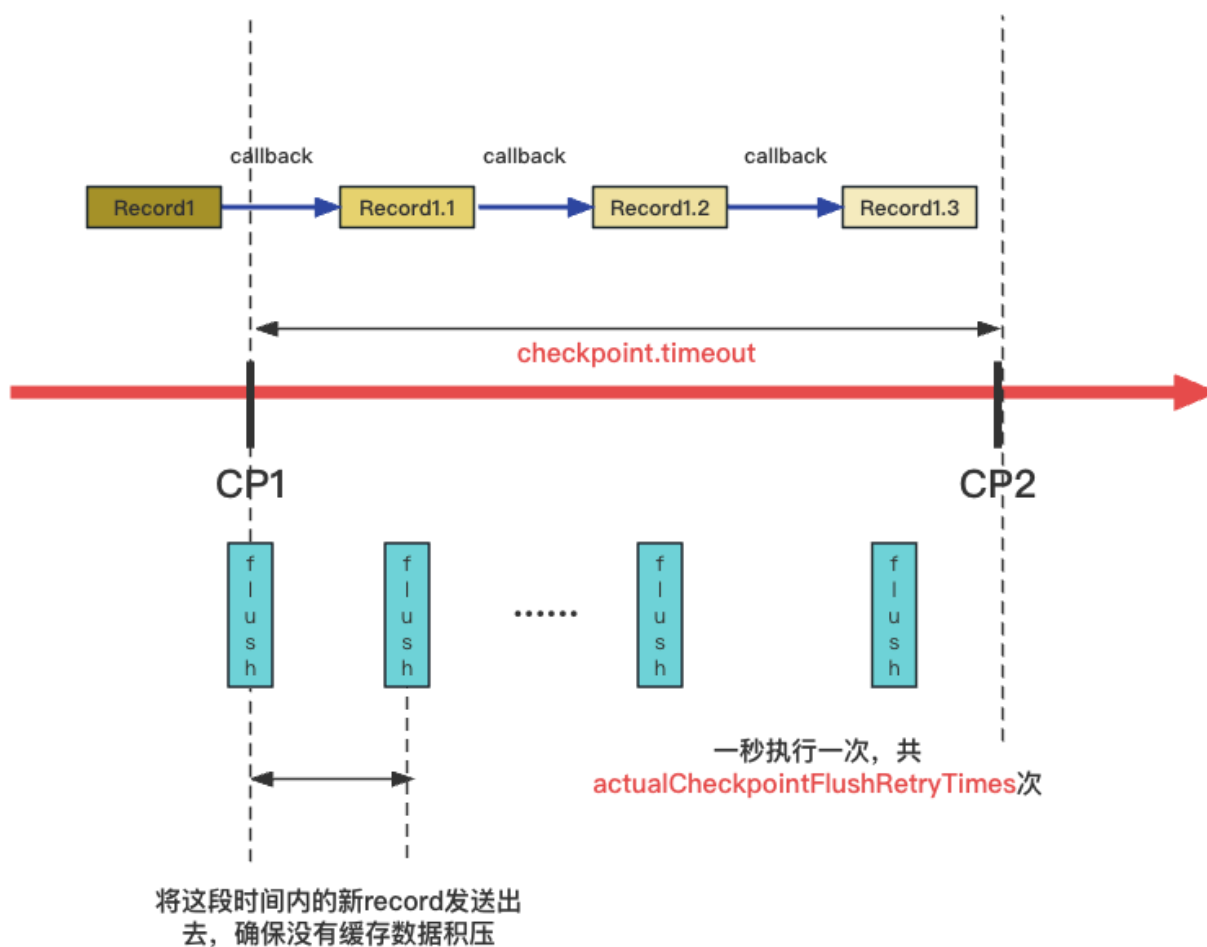
三、有关kafka参数和RetryableProducerCallback参数规范

下图红色时间轴为Flink时间，模拟一个checkPoint完整执行过程。

时间轴上为FlinkKafkaProducer中invoke主函数执行过程，对于Record1这里考虑极端情况，在CP1开始时，Kafka Broker不可用。首先该发送函数线程，会等待`delivery.timeout.ms`时间，在超时失败后触发回调函数，选择其它partition，考虑极端情况会重试`actualCallbackRetryTimes`次。

时间轴下方FlinkKafkaProducer中snapShot中执行流程，会每隔1s进行一次flush动作，除了第一个flush动作时将CP1前所有数据确保发送完毕，后面的flush动作是把CP执行过程中，callback中重发的数据进行一次全量发送下游。执行次数由`actualCheckpointFlushRetryTimes`指定。

record发送失败，重试`actualCallbackRetryTimes`次默认重试3次可以
每次重试超时时间通过`delivery.timeout.ms`指定默认120s



参数合理性设置

为了保证所有流程严格正确执行，参数设置必须符合下面要求

$\text{actualCallbackRetryTimes} * \text{delivery.timeout.ms} < \text{actualCallbackRetryTimes} < \text{checkpoint.timeout} * [\text{factor: default } 0.8]$

下面以RTLTP为例

`actualCallbackRetryTimes` 默认值3

`actualCallbackRetryTimes` 默认值120

`delivery.timeout.ms` kafka自身默认120,000 可以缩小为30,000

`checkpoint.timeout` rtlp设置为60s 可以改为180s

此时考虑到最坏情况. $90s < 120 < 180s * 0.8$ ，整个流程能够确保完整运行。

flush注意事项

在CP阶段的多次flush，会导致CP时间增加，例如执行30次flush，但实际时间可能大于30s达到60多s。主要原因如下

flush 是将缓存区的数据强制推送发送，调用flush时，会清空缓存区内存，调用 **awaitFlushCompletion** 时需要等待缓存区清空，这里会造成线程的堵塞。

```
public void awaitFlushCompletion() throws InterruptedException {
    try {
        Iterator var1 = this.incomplete.copyAll().iterator();

        while (var1.hasNext()) {
            ProducerBatch batch = (ProducerBatch)var1.next();
            batch.produceFuture.await();
        }
    } finally {
        this.flushesInProgress.decrementAndGet();
    }
}
```

正常情况下清空缓存区操作通过参数配置实现：

1.batch.size

离线缓存达到该size时执行一次flush

2.linger.ms

达到该时间间隔时，执行一次flush

所以**linger.ms**现在默认设置为0，可以缓解多次flush执行的阻塞问题

无标签